Ambush! — Game Design Document

Final version · Noah · ELVTR Game Development course, Assignment 1 · Due Tuesday, Oct 20, 2026

Contents

1. Overview
2. Setting and Tone
3. Core Loop (3.1 Screens between battles)
4. Core Mechanics (4.1 Command model through 4.13 Asset roster)
5. Resources and Scoring
6. Attack Mode
7. Prototype Scope
8. Pillar 1 — The AI Dev Pipeline
9. Pillar 2 — Technical Requirements and Constraints
10. Pillar 3 — Token Budgets and Projections
11. Runtime Agent — Front
12. Runtime Agent — Commander

Assignment requirements map

Requirement	Where it is covered
1. AI dev pipeline	Section 8 (spec to agent to JSON to Unity), plus the runtime agents in sections 11 and 12
2. Technical requirements and constraints	Section 9; biggest technical constraint and how it is handled in 10.2
3. Token budgets and projections	Section 10 (baseline, projection by phase, cost), plus the runtime budgets in sections 11 and 12
Named agents and their roles	10.2 (six build agents), 11 (runtime agent Front), and 12 (runtime agent Commander)

1. Overview

Ambush! is a real-time tactical command game set in a near-future war fought across the real map of Ukraine. The player commands squads, vehicles, and drones to hold a defensive line against enemy assaults, and can also go on the attack using the same mechanics with roles reversed.

- **Genre:** real-time tactics, squad command (no micromanagement)
- **Camera:** angled three-quarter top-down view (StarCraft-style), roads and lines running diagonally
- **Visual reference:** Company of Heroes 2/3 terrain density, rendered in a readable, slightly cartoonish pixel-art style
- **Platform (prototype):** PC/Mac, built in Unity 6 (URP)
- **Core fantasy:** a commander making hard choices with limited men and supplies, where every loss and every civilian casualty counts

2. Setting and Tone

A near-future conflict fought across the real map of Ukraine and its major towns, modeled on present-day trench and drone warfare: tree lines, trenches, ruined buildings, and contested villages. The weapons are current-day equipment plus near-future concepts such as exosuits, advanced drones, and electronic warfare.

The tone is grounded and serious. The game wants the battlefield to feel like what is happening in real wars today: drones overhead, jamming, netting over trenches, small units fighting for a few hundred meters of ground. Civilian casualties are tracked and count against the player, which gives every use of heavy firepower real weight.

3. Core Loop

One turn = one battle cycle.

1. **Prepare at HQ:** refill squads, upgrade units, assign specialists, plan how to spend the current supply pool.
2. **Deploy and fortify:** position squads and vehicles on the line; sappers build fortifications.

3. **Fight (real time):** the enemy attacks the line (or the player attacks). The player issues commands; units carry them out on their own. Drones and reserve munitions are called in as needed.
4. **Resolve:** territory gained or lost, casualties tallied (soldiers and civilians), veterancy updated, wiped-out units removed.
5. **Resupply:** command delivers supplies on a fixed schedule. Return to step 1.

The tension of the loop: spending heavily to win one battle leaves less for the next.

3.1 Screens between battles

Several screens sit between battles and the interactive battle screen. The central one is the **strategic map**: the player selects where the next attack will be, or sees where the next defensive stand will be.

Long-range drone strikes into enemy territory are launched from the strategic map, to scout or weaken the enemy's next attack before the battle begins.

The strategic map is the **real map of Ukraine with its major towns**. After each battle, the runtime agent Front (see 11) decides how much ground changes hands and where the next battle is fought.

Strategic sites in Russia: the strategic map extends into Russian territory, where strategic sites can be attacked with **long-range assets**: long-range drones, bombs, and missiles. Hitting these sites weakens the enemy's coming attacks.

Site	Effect when hit
Ammo depot	Shorter enemy bombardments in the next battles
Airfield	Less enemy air scouting and bombing
Refinery	Fewer enemy vehicles in the next waves (less fuel)
Rail hub	Slower enemy reinforcement; fewer waves per battle
Command post	Enemy Commander plans less well for a time (slower, simpler tactic changes)

Effects wear off over several battles as the enemy repairs the site.

4. Core Mechanics

4.1 Command model

The player gives each unit general orders, **forward, hold, back, or retreat**, with a direction. Units carry out the order on their own: how well they take cover, maneuver, and fight depends on their veterancy and battlefield intelligence. There is no direct control of individual soldiers. The player's decisions are what to send in first, where to hold and set up, and when to advance troops forward to take ground.

Camera and battlefield map: the battle view uses one fixed zoom level. Pressing a key (e.g. the spacebar) opens a map of the entire battlefield that shows **fog of war**: areas your units or scout drones can currently see, and areas that are hidden. Concealed enemy units in hidden areas do not appear until they are spotted.

4.2 Squads

- The basic unit is a **squad of five soldiers**.
- Every soldier model has **its own health bar**. When soldiers die, the squad shrinks and gets weaker.
- Squads start generic and are **upgraded with abilities**. An upgrade turns one soldier into a specialist (e.g. sapper, demolitions, heavy weapons, drone operator), with a **maximum of one specialist per squad**.
- The rest of the squad supports the specialization. Example: a heavy weapons upgrade gives the squad a heavy machine gun and anti-tank capability.

4.3 Unit roles

Every squad starts as **riflemen**. Riflemen are then upgraded into one of the other roles.

Role	How you get it	What it is
Rifleman	Base unit	Standard infantry; the starting point for every squad
Exosuit trooper	Upgrade from Rifleman	Heavy infantry / shock troops: better infantry with upgraded stats (armor, health, firepower)
Heavy weapons	Upgrade from Rifleman	Machine guns and anti-tank weapons; suppression and the main infantry answer to armor

Role	How you get it	What it is
Drone operator	Upgrade from Rifleman	Short-range drone strikes and recon from the back line
Engineer	Upgrade from Rifleman	Sapper work: builds fortifications and netting, demolitions, electronic warfare

Each role is a **single upgrade** of a rifle squad.

Tech tree: tech advances with time only, at no supply cost; the player picks which branch to advance. There are four branches, each with four tiers.

Tier	Drones and missiles	Infantry	Vehicles	Economy
1	Drone operator role; faster drone loading; ground drone with a machine gun or recovery kit	Heavy weapons and Engineer roles	Cage and standoff armor	Cheaper squad refills
2	Fiber-optic upgrade; +1 drone per team; anti-tank weapon for ground drones	Body armor (more soldier health)	Air defense (Gepard / Tunguska)	Faster resupply schedule
3	Loitering munitions; tighter strike area	Better cover use and maneuvering	Heavy tanks and self-propelled howitzers	Larger supply pool
4	Swarm control (+2 drones per team); long-range strikes	Exosuit trooper role	EW truck	Long-range strike budget on the strategic map

4.4 Vehicles

Vehicles are standalone units with their own crew (crew size varies by vehicle). Squads can ride inside armored transports and trucks. The roster is kept small on purpose: eight vehicles across three classes (heavy armor, light armor, light/transport). See 4.13 for the full list.

4.5 Hard counters

Units have clear strengths and weaknesses. Regular rifle squads **cannot effectively damage tanks**; armor must be handled with anti-tank weapons (e.g. a heavy weapons squad) or drones. Picking the right unit for the threat is a core skill.

4.6 Veterancy

- Squads and vehicles gain veterancy the longer they survive in combat.
- Higher veterancy means smarter battlefield behavior (positioning, target choice), better survival, and more effective fighting.
- **Reinforcing a squad keeps its veterancy.** A unit that is completely wiped out is gone, along with all its veterancy.

4.7 Retreat

There is no morale system; the player decides when to retreat. A retreated unit is **out for the rest of that battle** and fights another day. Injured squads must either keep fighting or fall back to a rear position and fight from there. There is no reinforcing during a battle.

4.8 Drones and reserve munitions

- The player has a **limited arsenal** of long-range drones and reserve munitions that can be deployed mid-battle. They fly in from off-map and strike enemies.
- Drones **do not always hit** and can be countered.
- **Short-range drones** are launched by drone teams on the map. These teams operate concealed from the back line, strike at long range, and fight with rifles if engaged. They can be found and destroyed.
- Design intent: drones are powerful but scarce, so battles are won by combining them with squads and vehicles.

Drone types: drone combat is a core pillar, with a variety of drones at each range. All drone capabilities are unlocked and improved through the tech tree.

Drone	Range	Main use	Targets
FPV kamikaze	Short	The most-used drone on the battlefield; one-way strike	Infantry, light vehicles, positions

Drone	Range	Main use	Targets
Scout drone	Short to medium	Spots enemy units and guides strikes	Reveals targets; no damage
Loitering munition	Medium	Circles an area, then dives on a target it finds	Vehicles, artillery, tanks
Long-range strike	Long (strategic map)	Strikes inside the enemy country	Logistics and assets behind the front

Fiber-optic upgrade: FPV and scout drones can be upgraded to fiber-optic control, which makes them immune to EW jamming but limits their range and cable length.

Ground drones: unmanned ground vehicles controlled by drone teams. At tech tier 1 a ground drone is fitted with either a **machine gun** or a **recovery kit** that pulls your own losses, or enemy losses, off the field. Tech tier 2 adds an **anti-tank weapon**.

Player-controlled drone strikes: drone warfare is directly controlled by the player.

1. **Select:** click a drone team, choose which drone to fire, and see its range.
2. **Ready:** a new team loads one drone at a time and fires one at a time. Operator skill, veterancy, and tech combined decide how fast a team readies drones and how many it can pilot at once.
3. **Launch:** click Launch Drone near the team, then pick the area. A highlighted zone appears; the team flies the drone over it and tries to hit a vehicle or target inside. Accuracy improves with veterancy and the tech tree.
4. **Multi-launch:** to fire several drones, select several drone teams and load their drones onto the cursor, one per team at first. As skill and tech rise, each team can carry more. The player chooses how many to launch at the area, to overwhelm air defenses and electronic jamming.

Long-range strikes are the logistics side of the war. They hit targets deep in enemy territory to scout the enemy's next attack or weaken it before it arrives, and to bring the war closer to an end.

4.9 Drone counters

All six real-world counters are included:

1. **Electronic warfare jamming:** an EW specialist creates a zone where drones lose control.

2. **Shooting them down:** small arms at close range with poor odds; dedicated anti-air is much better.
3. **Physical protection:** sapper-built netting over trenches and roads; cage armor on vehicles.
4. **Concealment:** units under tree lines or in trenches are harder to spot.
5. **Interceptor drones:** drones that hunt enemy drones.
6. **Striking operators:** only possible against on-map short-range drone teams; long-range arsenal drones have no operators to hit.

4.10 Terrain and fortifications

- Maps include buildings, tree lines, and trenches.
- Some maps have **fixed, indestructible fortifications**.
- The player can always **build new fortifications**, which are destructible. Sappers/engineers build them.

4.11 Level design

Maps are based on **real cities and towns, referenced from OpenStreetMap**, and recreated in pixel art. Real street layouts, rivers, rail lines, and building blocks give each map a believable shape and natural defensive lines.

Workflow:

1. **Reference:** pick a real location and capture its layout (roads, buildings, tree lines, open fields, water).
2. **Abstract:** simplify it to a grid. Mark the defensive line, approach routes, chokepoints, cover, and fixed fortifications.
3. **Layout file:** describe the map as a JSON layout (tiles, buildings, trenches, spawn and objective zones).
4. **Build:** the level agent turns the layout into a Unity tilemap using the pixel-art tile set.
5. **Play-test:** check sight lines, cover, and attacker routes; adjust and repeat.

Each map defines: the friendly defensive line, enemy approach routes, fixed indestructible fortifications (if any), spaces where new fortifications can be built, and civilian areas (where casualties count against the score).

Real locations: battle maps are drawn from real Ukrainian towns and their surroundings, chosen by where the front line sits on the strategic map.

Randomness: each map has a fixed real-world base layout, with randomized variation layered on top so no two battles play the same:

- **Damage state:** which buildings are intact, damaged, or rubble, and where craters are
- **Cover:** density and extent of tree lines, hedgerows, and wrecked vehicles
- **Fortifications:** which pre-built trenches and positions exist at the start (fixed fortifications stay fixed)
- **Civilian areas:** where civilians are still present
- **Enemy approach:** which routes the waves use, and what each wave contains

Random variation is generated from a seed within rules set in the layout file (e.g. the defensive line and key roads never change), so maps stay fair and play-testable.

4.12 Enemy assault structure

An enemy attack comes in **waves**. The number varies with how the battle is going; some battles are a single wave.

Each wave follows a set order:

1. **Bombardment and scouting:** artillery or bombing, with drones or aircraft scouting the player's line.
2. **Heavy vehicles:** armor leads the push.
3. **Infantry:** troop transports bring squads forward, or infantry assault on foot.

This order teaches the player to read the attack: survive and stay concealed during the bombardment, hold anti-tank weapons for the armor, then hold the line against infantry.

How a battle ends:

- **Defeat:** the enemy breaks through and forces the player's overall force to retreat. Territory is lost.
- **Victory:** the player holds the line and pushes the attackers back.

4.13 Asset roster

Everything the player can deploy. The roster is kept small on purpose. Vehicles are real-world models used in the war in Ukraine.

Infantry squads (5 soldiers, one specialist max)

Squad	Specialist	Strengths	Weaknesses
Rifle squad	none	Cheap, flexible, holds trenches	Cannot hurt armor
Exosuit squad	Exosuit trooper	Tough shock troops, upgraded stats	Expensive
Heavy weapons squad	MG / anti-tank gunner	Suppression, kills vehicles	Slow, needs cover
Drone team	Drone operator	Short-range drone strikes and scouting from the back line; ground drones (machine gun, recovery, anti-tank)	Weak if found
Engineer squad	Sapper / demolitions / EW	Builds fortifications and netting, mines, jams drones	Poor fighters

Vehicles (three classes, own crew)

Each side fields **its own separate vehicles** with similar stats, so the two armies look and feel distinct while staying balanced.

Class	Vehicle	Role	Player side	Enemy side
Heavy armor	Heavy tank	Slower, bigger firepower, longer range	Leopard 2, Challenger 2	T-90
Heavy armor	Main battle tank	Faster, shorter range; leads assaults and flanks	M1A2 Abrams	T-72
Light armor	Self-propelled howitzer	Long-range bombardment from behind the line	PzH 2000	2S19 Msta-S
Light armor	Infantry fighting vehicle	Carries one squad, autocannon support	M2 Bradley	BMP-3
Light armor	Anti-air tracked vehicle	Cannon platform that shoots down drones	Gepard	Tunguska

Class	Vehicle	Role	Player side	Enemy side
Light / transport	Armored transport	Carries two squads, lightly armed	MRAP	BTR-82A
Light / transport	EW truck	Wide-area drone jamming; unarmed, priority target	EW truck	EW truck
Light / transport	Technical	Fast armed pickup for raids and quick anti-drone fire	Armed pickup	Armed pickup

Vehicle armor upgrades: all heavy and light armor can be upgraded with **cage armor or standoff armor**, giving extra protection against drones and anti-tank weapons.

Arsenal (limited, called in mid-battle)

Asset	Effect
Long-range strike drones	Fly in from off-map and attack a target; can miss or be countered
Interceptor drones	Hunt enemy drones
Artillery barrage	Area bombardment; risks civilian casualties
Reserve munitions	Extra ammo and anti-tank rounds for the line

5. Resources and Scoring

Supplies (what you fight with) and score (how you are judged) are kept separate.

5.1 Supplies

- A **finite supply pool** from military command funds units, upgrades, refills, drones, and munitions.
- **Resupply arrives on a fixed schedule.**
- Squads are **refilled between battles**, which costs supplies.
- Overspending in one battle means weaker or more expensive options in the next.

5.2 Score

- **Territory** gained or lost.
- **Casualties:** your own soldiers lost, **plus civilian casualties you cause**, including from indirect fire such as artillery, bombing, and drone strikes. These are tallied after each battle.

6. Attack Mode

The game is defense-focused, but the player can also attack. Attack mode uses the same systems with roles reversed: the player leads the assault and the computer defends its line.

Assault sequence:

1. **Scouting:** drones or aircraft locate enemy positions, fortifications, and anti-tank threats.
2. **Bombardment:** artillery, bombing, and drone strikes soften the enemy position to break their line. Civilian casualties caused here still count against the score.
3. **Main force:** the player commits a chosen set of assets. They can lead with vehicles or with infantry, then adapt from there.

Winning an attack: break through the enemy line and force them to retreat, gaining territory. **Losing:** the assault stalls or is pushed back, at a cost in men and supplies.

7. Prototype Scope

A working Unity 6 prototype already exists from the earlier version of the concept (a unit moving on a diagonal road, player input, a basic win state, HUD). The next prototype step is to prove the core command loop:

- One defensive line on one small map with a tree line and a trench
- Two or three squads of five soldiers with per-soldier health bars
- One enemy assault wave
- Commands: forward, hold, back, retreat, each with a direction
- One drone strike with a limited count
- Basic scoring: territory held and casualties

Not in the first prototype: vehicles, veterancy, specialist upgrades, attack mode, the full drone counter set, art beyond placeholder shapes, and the runtime agents (Front and Commander), which are stood in for by their rule-based fallbacks.

8. Pillar 1 — The AI Dev Pipeline

The game is built mainly with **build-time AI agents**. Two runtime agents run while the game is played: Front runs the campaign between battles (section 11), and Commander sets battle tactics for both sides (section 12). This section maps exactly how a design decision becomes a working object in Unity.

1. **Spec (raw text):** this GDD plus a project `CLAUDE.md` file in the repo. Together they are the contract: what to build, what is out of scope, and project conventions.
2. **AI node (agent):** Claude Code reads the spec, plans first (plan mode), and breaks a feature into steps. The human approves the plan.
3. **Translation (JSON):** each step becomes a tool call sent as structured JSON over the Model Context Protocol (MCP) — e.g. create a GameObject, add a component, write a C# script, set an Inspector value.
4. **Bridge:** the MCP for Unity server (Python, stdio transport) receives the JSON and relays it to the Unity Editor plugin on a local port.
5. **Engine (Unity 6):** the plugin executes the command — scenes, objects, and scripts appear in the project. Unity returns results and console errors as JSON back to the agent.
6. **Verify:** the agent reads the console, fixes compile errors, and the human play-tests. Accepted work is committed to GitHub.

Data-driven content: unit stats (health, damage, cost, veterancy bonuses), vehicle classes, and drone counters live in JSON data files. Agents generate and edit these files; Unity loads them at startup. Designers change balance without touching code.

Guardrails at each handoff: JSON is schema-validated before Unity loads it (cheap, deterministic); a reviewer pass checks generated content against the GDD (e.g. no unit kills a tank without anti-tank weapons); every agent change goes through plan approval and Git, so it can be rolled back.

Known silent-failure points: Unity not running or the bridge session inactive (calls return zero instances); the agent tool list only loads at Claude Code startup; malformed JSON data files. Each has a check in the workflow.

9. Pillar 2 — Technical Requirements and Constraints

Development environment: MacBook Pro, Apple M5 Pro, 24 GB RAM, macOS; Unity 6 (URP), new Input System; Claude Code with the MCP for Unity bridge; Python 3.12; Git/GitHub.

API and usage limits: development runs on a Claude subscription with rolling usage limits rather than per-call billing. Heavy agent sessions can hit those limits, so large features are scoped into plan-first chunks. Any scripted pipeline using the API directly has a monthly spending cap set in the console.

Context window: the agent can only hold a limited amount of project text at once. Constraints that follow:

- `CLAUDE.md` stays lean (rules and current scope only); the full GDD is referenced, not pasted.
- Unit and balance data live in small JSON files the agent reads on demand.
- One feature per session; long sessions are compacted or restarted.

Processing latency: each build agent step takes seconds to minutes, and each MCP round trip to Unity adds time; this is acceptable at build time. At runtime, an LLM call takes a second or more, so **no LLM call ever sits in the frame loop** (16 ms target). Front runs between battles (section 11). The Commander (section 12) runs in the background during battles and only sets high-level plans; units are moved every frame by conventional behavior trees and never wait for it.

Runtime game constraints: unit behavior uses conventional game AI (state machines / behavior trees) so dozens of units can run in real time on modest hardware.

Runtime models: both runtime agents (sections 11 and 12) use a small open-source model running locally through LLMUnity, not a cloud API. This avoids shipping an API key in the game, per-player API costs, and an internet requirement. The trade-offs are higher memory use (about 4–8 GB), a larger download, weaker reasoning than a cloud model, and GPU competition during battles. Every runtime agent has a rule-based fallback.

10. Pillar 3 — Token Budgets and Projections

This projection is made before development starts, as Pillar 3 requires, and it is calibrated against real Claude Code usage measured with `ccusage`.

10.1 Baseline (measured)

- Total measured usage to date: **~345M tokens, $286 API-equivalent**, almost all on Opus.
- **~97% cache reads** (334.9M of 345.2M): agents rereading context. That keeps the effective cost under $1 per million tokens.
- Heavy build days: **70–105M tokens, $50–76 API-equivalent** each.

10.2 Agent roster and model tiers

Agent	Input	Output	Model
HQ (architect)	GDD, current build state	JSON task spec: target files, acceptance test, assigned agent	Fable or Opus
Forge (gameplay)	HQ task spec plus only the files it names	C# scripts in Assets/Scripts that compile cleanly	Haiku
Ledger (data)	Task spec, unit/vehicle/drone JSON	Validated JSON stat files	Haiku
Painter (assets)	3D model renders, style guide	Pixel-art sprites and tilesets at a fixed angle	Haiku
Cartographer (levels)	OpenStreetMap reference, map spec	JSON grid layout and Unity tilemap	Haiku
Sentry (QA)	Console log, build output, GDD rules	Pass/fail report with bugs routed to Forge	Haiku

Agent constraints:

- **HQ** never writes game code; a task may touch at most 10 files.
- **Forge** edits only files named in the spec, never JSON data or scenes, and fixes compile errors before reporting back.
- **Ledger** edits only JSON data files; every file must pass schema validation.
- **Painter** uses one cohesive style and palette; only licensed models cleared for commercial use.
- **Cartographer** uses OpenStreetMap only; the defensive line and key roads stay fixed, randomness is seeded.
- **Sentry** reports and never edits files.

What the player sees when each agent fires:

- **HQ:** nothing directly; it decides which features exist and when.
- **Forge:** units obeying forward, hold, back, and retreat, drones flying to the highlighted area, the score tally.
- **Ledger:** how hard a T-90 hits, how long a squad survives, what a refill costs.
- **Painter:** every soldier, vehicle, drone, and building on screen.
- **Cartographer:** the battlefield itself: streets, tree lines, trenches, and the line to hold.
- **Sentry:** a game that runs without crashes and follows the GDD rules.

Biggest technical constraint: Haiku running the complex code. Five of six agents run on Haiku to keep token cost down, but Haiku is weaker on complex systems like squad AI and drone targeting. **How it's handled:** HQ breaks every feature into small, tightly scoped tasks with an acceptance test; Forge sees only the files it needs; Sentry checks each result against the GDD; and if a task fails twice, HQ escalates it to Opus.

Human stage gates. Agents never run unattended across a phase. The developer approves and redirects at fixed points, which also caps token use:

1. **Plan gate:** HQ's task list for a phase is approved by the developer before any agent starts.
2. **Budget gate:** each phase has a token ceiling from the table above; if it is reached, all agents stop and wait for the developer.
3. **Phase gate:** at the end of each phase the developer play-tests the build and signs off before the next phase opens.
4. **Failure gate:** if a task fails after escalation to Opus, it goes to the developer, not back into the loop.

The architect does the least typing and the most reasoning, so the expensive model handles a small share of the tokens. Delegated agents get short, focused contexts to keep cache reads down.

10.3 Projection by phase

Assumes one to three heavy days per phase, with architect work about 25% of tokens and delegated work about 75%.

Phase	Heavy days	Est. tokens
1. Prototype (core command loop)	1–2	35–100M

Phase	Heavy days	Est. tokens
2. Squad systems	1–2	35–100M
3. Vehicles and counters	1–2	35–100M
4. Drones and drone counters	2–3	70–150M
5. Economy and scoring	1	35–50M
6. Level design and maps	1–2	35–100M
7. Data, assets, balance	1–2	35–100M
8. QA and bug fixing	1–2	35–100M
9. Runtime agents (Front and Commander)	1–2	35–100M
Total	**10–18 (fewer in parallel)**	**~350–900M**

Agents by phase. HQ runs throughout as the single point of contact.

1. HQ splits section 7 into tasks; Forge builds the command system, squad movement and per-soldier health; Ledger writes the first rifleman JSON; Sentry checks the loop runs.
2. Forge builds squad AI, specialist upgrades, retreat and veterancy; Ledger adds the four specialist roles; Sentry tests the hard-counter rules.
3. Forge builds vehicle movement, armor and anti-tank damage; Ledger writes all 16 vehicle entries; Painter renders vehicle sprites.
4. Forge builds drone flight, miss chance and all six counters; Ledger writes drone and EW data; Sentry verifies each counter.
5. Forge builds the supply pool, resupply schedule and score tally; Ledger sets costs and resupply amounts; Sentry checks the score math.
6. Cartographer converts OpenStreetMap references to JSON grids and builds tilemaps; Painter renders the tileset; HQ reviews layouts.
7. Painter renders and organizes all unit sprites; Ledger tunes stats from play-tests; Sentry validates every JSON file.
8. Sentry reads console errors and checks output against the GDD; Forge fixes what it reports; HQ signs off.
9. Forge integrates the local model through LLMUnity, writes the Front and Commander prompts and the rule-based fallback planners; Ledger defines the JSON order schemas; Sentry tests that invalid or slow output always falls back.

The token total is the same whether phases run in sequence or in parallel; only the calendar changes. Level design, assets, and data/balance depend on nothing but the GDD, so they run alongside the code phases from the start. The code phases are sequential: the prototype first, then squads, vehicles, drones, and the runtime agents in order, because each builds on the one before. QA runs continuously as each phase lands.

Multi-agent overhead: each agent loads its own context, so parallel agents multiply cache reads. The upper end of the range covers this.

10.4 Cost

- **Actual cost during development:** a flat Claude Max subscription per month. The real limit is the plan's rolling usage caps, not a bill.
- **Why the measured count is so high:** about 97% of tokens are cache reads, meaning agents rereading context every step. The estimates above assume lean days of 35–50M, about half the measured rate, achieved by:
  - a one-page CLAUDE.md, since it loads every turn
  - each agent seeing only its own files
  - a fresh session per task instead of long-running chats
  - small task specs from the architect so Haiku agents don't explore the codebase
  - plain scripts for repeat jobs like sprite rendering and JSON validation
- **API-equivalent cost (for commercial viability):** with the architect on Fable/Opus (~25% of tokens) and every other agent on Haiku (~75%), the blended rate is roughly $0.33 per million, putting the full build at about **$115–300**.

Cost controls: top model for the architect only; lean context files (Pillar 2); plan before every build to avoid wasted runs; track `ccusage` after each phase and revise this projection after the prototype.

11. Runtime Agent — Front

The first runtime agent is **Front**, the campaign agent: it runs the war on the strategic map between battles and adds randomness to the campaign, so no two playthroughs follow the same path.

Item	Detail
Input	Strategic map state (real Ukraine map, major towns, current front line), the battle result and score, territory held by each side
Output	JSON: territory gained or lost, the new front line, the location of the next battle (attack or defense), and the battle map spec for the Cartographer's layout
Model	Open-source model (e.g. Qwen or Llama, 3–8B parameters, quantized) running locally on the player's computer through LLMUnity

How it decides:

1. **Territory swing:** the battle score sets the direction and base size of the swing; the agent adds a random factor so results are not fully predictable. A narrow win may gain little ground; a collapse may lose a whole town.
2. **Next battle:** the agent picks where on the front the next fight happens, using the real towns, roads, and rivers near the new line.
3. **Map variation:** it chooses which real town the battle map is drawn from and sets the seeded randomness for that map (damage, cover, civilian areas, enemy routes, see 4.11).

Guardrails: the output is schema-validated before the game uses it; the territory swing is clamped to a maximum per battle; the next battle must be on or next to the current front line; if the local model fails or is too slow or the output is invalid, the game falls back to a rule-based result from the score and a seeded random roll, so the campaign never stalls.

What the player sees when it fires: after the results screen, the strategic map updates. The front line moves, towns change hands, and the next battle location is marked.

Requirements: the model ships with the game and runs on the player's machine, so there is no API key, no server, and no internet connection needed. It needs roughly 4–8 GB of memory and adds a few GB to the download. Machines that can't run it use the rule-based fallback.

Token budget: about 5K tokens in and 1K out per call, one call per battle. Because the model runs locally, there is no per-call cost; the budget limits wait time instead. One call takes a few seconds and is hidden behind the results screen.

12. Runtime Agent — Commander

The second runtime agent controls battle tactics for both sides. It is split into two layers so the game stays fast:

Layer	What it does	Runs on	How often
Commander (LLM agent)	Big decisions: plans enemy waves, changes enemy tactics when the player's line holds, turns the player's general orders into a plan for each squad	Same local open-source model as Front	At each wave start and every 20–30 seconds, or when something major happens (a squad wiped out, the line broken)
Unit AI (behavior trees)	Moment to moment: moving, taking cover, picking targets, firing	Unity, no LLM	Every frame

Enemy side: at the start of each wave the Commander reads the map, the visible player positions, and the wave budget, and outputs a plan (routes, which units lead, where to bombard). It revises the plan when the attack stalls or breaks through.

Player side: when the player gives forward, hold, back, or retreat with a direction, the Commander turns it into concrete goals for each squad (which tree line to occupy, which building to clear), adjusted for veterancy. The behavior trees carry those goals out.

Output: JSON orders per squad or vehicle (destination, stance, priority target). The behavior trees only accept orders that are valid on the map.

Guardrails: the game never waits for the Commander. Units keep executing their last orders until a new plan arrives. If a call fails or the output is invalid, a rule-based planner takes over. The enemy Commander only sees what its own units have spotted, so it obeys the same fog of war as the player.

What the player sees when it fires: the enemy shifts its attack to a weak point after a failed push; your squads spread across the right cover after a single order.

Token budget: about 4K tokens in and 500 out per call, roughly 20–40 calls per battle (about 100–180K tokens per battle). Running locally there is no per-call cost, but each call competes with the game for the graphics card. Calls run in the background at low priority,

and the Commander calls less often on slower machines; below a minimum spec, the rule-based planner runs instead.